LAPORAN PRAKTIKUM

TUGAS – Praktik 6

(TEKNOLOGI IOT II)



Disusun Oleh :

Nama : Feni Afelia
Kelas : 3A
NPM : 25782011

Dosen :

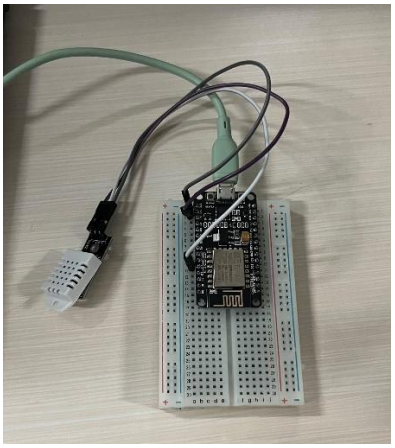
1. Dr. Ir. Septafiansyah Dwi Putra
2. Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG 2026

➢ HASIL PELAKSANAAN PRAKTIKUM

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Rangkaian :



Kode:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi (Gunakan Hotspot Pribadi Anda)
const char* ssid = "Feni";
const char* password = "12121212";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

// IDENTITAS KLIEN (Wajib Unik! Ganti dengan NIM Anda agar tidak bertabrakan)
const char* clientId = "ESP8266_LabIoT_25782011";

// Definisi Topik Publikasi Telemetri Mandiri
const char* topicPubTelemetry = "kampus/labiot/25782011/telemetri";

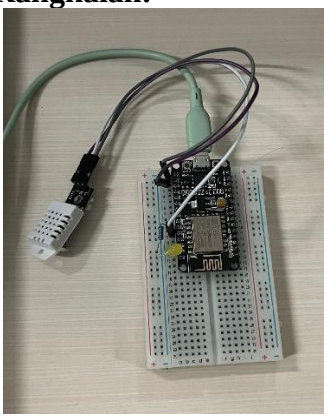
// Konfigurasi Pin Sensor DHT
const byte dhtPin = 2;   // Pin D4 (GPIO 2)
#define DHTTYPE DHT11
```

Hasil Output:

```
[11:58:07.912] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
[11:58:09.231] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
[11:58:10.965] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.70}
[11:58:12.937] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.30}
[11:58:17.912] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
[11:58:22.993] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.40}
[11:58:27.952] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.40}
[11:58:32.909] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.40}
[11:58:37.909] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":65.20}
[11:58:42.709] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":65.00}
[11:58:47.947] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":65.10}
[11:58:52.964] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.10}
[11:58:57.964] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.70}
[11:58:57.964] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.80}
[11:59:52.934] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.60}
[11:59:07.939] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.80}
[11:59:12.942] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.80}
[11:59:12.917] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.80}
[11:59:22.960] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.90}
[11:59:27.908] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.90}
[11:59:27.908] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
[11:59:32.960] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.70}
[11:59:32.982] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.90}
[11:59:47.942] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.70}
[11:59:47.942] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
[11:59:52.944] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
[11:59:52.944] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.80}
[11:59:57.909] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
[12:00:02.935] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
[12:00:07.955] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.60}
[12:00:12.955] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.40}
[12:00:12.908] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.60}
[12:00:17.914] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
[12:00:22.934] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
[12:00:27.936] -> Telemetri Terkirim [kampus/LabIot/25782011/telemetri]: {"node_id":"ESP8266_LabIOT_25782011","suhu":33.40,"kelembapan":64.50}
```

6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Rangkaian:



Kode:

sketch_5.5.4.ino | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12E M...

sketch_5.5.4.ino.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "Feni";
const char* password = "12121212";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782011_Sub";

// Definisi Topik Subskripsi Perintah Mandiri
const char* topicSubCommand = "kampus/labiot/25782011/perintah";

// Konfigurasi Pin Indikator LED
const byte ledPin = 5; // Pin D1 (GPIO 5)

WiFiClient espClient;
PubSubClient client(espClient);

void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
```

Ln 44, Col 29 NodeMCU 1.0 (ESP-12E Module) on COM5 [not connected]

Hasil Output:

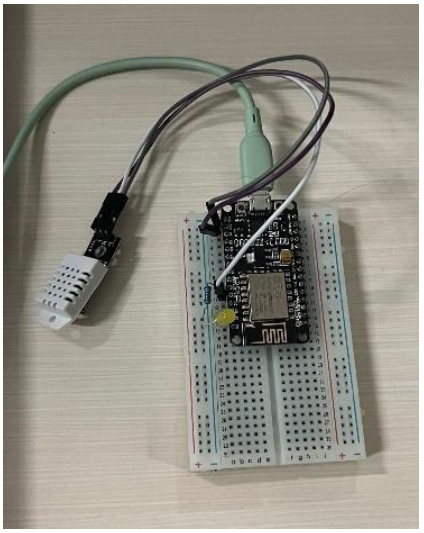
Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

```
12:26:57.531 -> ......
12:27:00.060 -> Wi-Fi Terkoneksi!
12:27:00.060 -> Mencoba koneksi MQTT... Terhubung!
12:27:00.512 -> Subskripsi Topik: kampus/labiot/25782011/perintah
```

6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

Rangkaian:



Kode:

Prak3_6.5.3 | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

Prak3_6.5.3.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "vivo Y21A";
const char* password = "lilis123";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782011_Full";

// Pembagian Topik Mandiri
const char* topicPublish   = "kampus/labiot/25782011/telemetri";
const char* topicSubscribe = "kampus/labiot/25782011/kendali";

// Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)
#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Siklus kirim 4 detik

void callback(char* topic, byte* payload, unsigned int length) {
```

Output:

Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

```
13:05:49.890 -> ....
13:05:51.393 -> Wi-Fi Terkoneksi!
13:05:51.393 -> IP Address: 192.168.109.134
13:05:51.433 -> Menghubungkan kembali ke Broker... Sukses Terhubung!
13:05:51.876 -> Subskripsi Topik: kampus/labiot/25782011/kendali
13:05:51.876 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.50,"kelembapan":64.00,"status_led":0}
13:05:55.877 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.50,"kelembapan":64.20,"status_led":0}
13:05:59.904 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.50,"kelembapan":64.20,"status_led":0}
13:06:03.911 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.50,"kelembapan":64.40,"status_led":0}
13:06:07.878 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.50,"kelembapan":64.30,"status_led":0}
13:06:11.881 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.50,"kelembapan":64.00,"status_led":0}
13:06:15.882 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.60,"kelembapan":64.00,"status_led":0}
13:06:19.906 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.60,"kelembapan":64.00,"status_led":0}
13:06:23.879 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.50,"kelembapan":63.80,"status_led":0}
13:06:27.883 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.60,"kelembapan":63.90,"status_led":0}
13:06:31.877 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.60,"kelembapan":63.80,"status_led":0}
13:06:35.902 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.60,"kelembapan":63.90,"status_led":0}
13:06:39.879 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.60,"kelembapan":63.90,"status_led":0}
13:06:43.914 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.50,"kelembapan":63.90,"status_led":0}
13:06:47.928 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.60,"kelembapan":63.90,"status_led":0}
```

6.5.4 Panduan Pengujian Diagnostik dengan MQTT Explorer

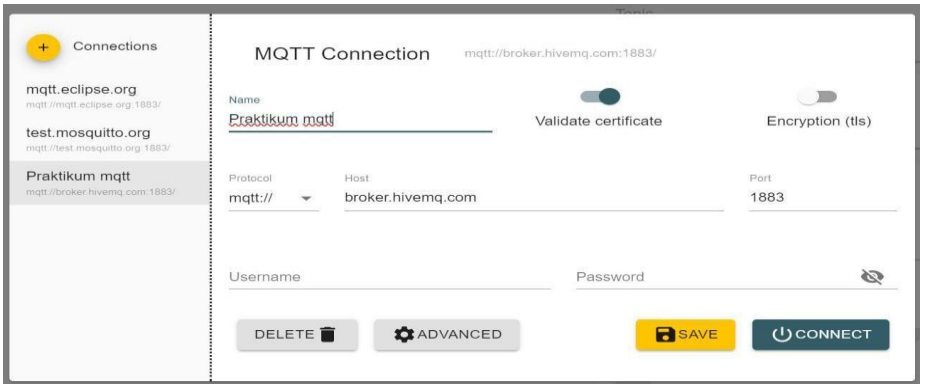
1. Jalankan Aplikasi MQTT Explorer pada laptop Anda.
2. Konfigurasi Koneksi Broker:

   Protocol: mqtt://

   Host: broker.hivemq.com

   Port: 1883

   Klik tombol Connect.

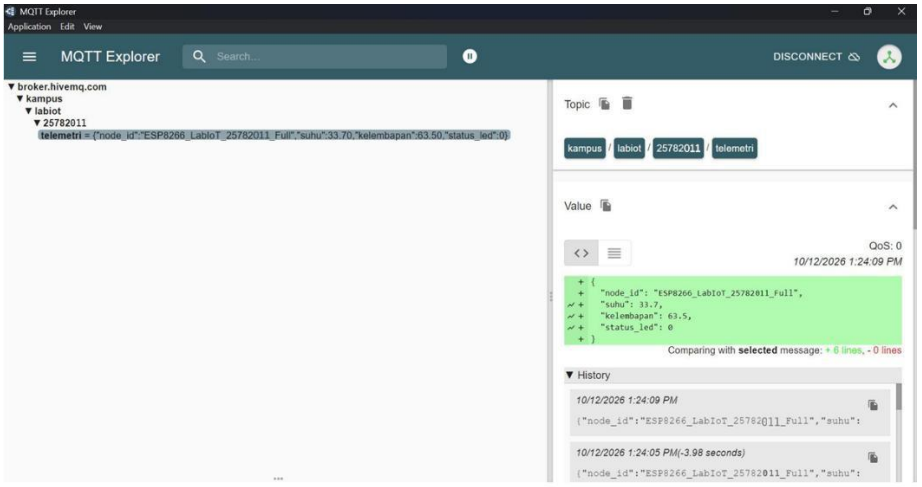


3. Memantau Data Masuk (Telemetry Monitoring):

Amati pohon hierarki topik (Topic Tree) pada panel sebelah kiri.

Temukan dan klik cabang topik Anda: kampus → labiot → nim_anda → telemetri.

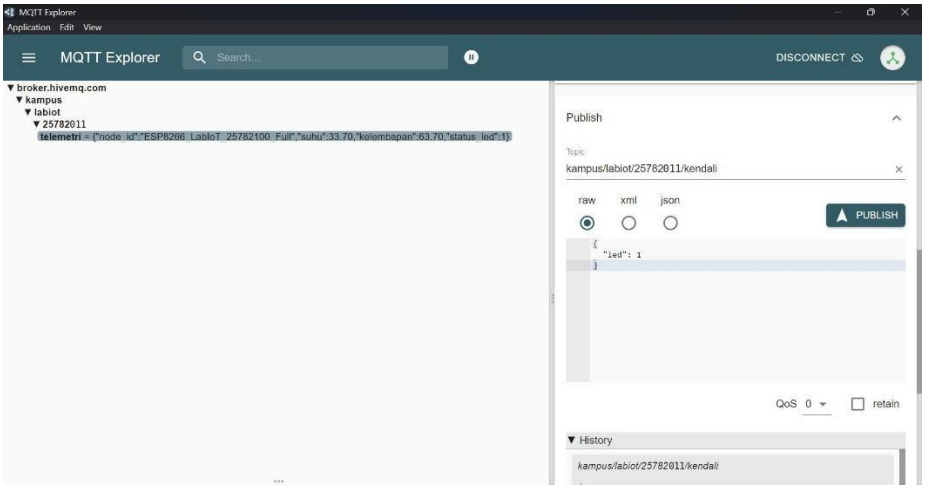
Anda akan melihat payload JSON diperbarui secara real-time setiap 4 detik. Klik pada nilai numerik untuk menampilkan grafik fluktuasi suhu dan kelembapan secara langsung.



Hasil output di Serial Monitor:

```
13:22:15.292 -> Menghubungkan kembali ke Broker.......
13:22:49.794 -> Wi-Fi Terkoneksi!
13:22:49.794 -> IP Address: 192.168.109.134
13:22:49.794 -> Menghubungkan kembali ke Broker... Sukses Terhubung!
13:22:57.701 -> Subskripsi Topik: kampus/labiot/25782011/kendali
13:22:57.701 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.20,"status_led":0}
13:23:01.748 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.10,"status_led":0}
13:23:05.731 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.10,"status_led":0}
13:23:09.745 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.10,"status_led":0}
13:23:13.704 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.10,"status_led":0}
13:23:17.729 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.00,"status_led":0}
13:23:21.708 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.00,"status_led":0}
13:23:25.734 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.30,"status_led":0}
13:23:29.734 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.20,"status_led":0}
13:23:33.708 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.30,"status_led":0}
13:23:37.732 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.30,"status_led":0}
13:23:41.708 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.60,"kelembapan":63.60,"status_led":0}
13:23:45.747 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":64.20,"status_led":0}
13:23:49.732 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.70,"status_led":0}
13:23:53.737 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.40,"status_led":0}
13:23:57.719 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.30,"status_led":0}
13:24:01.703 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.20,"status_led":0}
13:24:05.744 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.40,"status_led":0}
13:24:09.727 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.50,"status_led":0}
13:24:13.703 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.50,"status_led":0}
13:24:17.739 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.50,"status_led":0}
13:24:21.703 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.50,"status_led":0}
13:24:25.739 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.40,"status_led":0}
13:24:29.727 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.40,"status_led":0}
13:24:32.717 -> ----------------------------------------
```

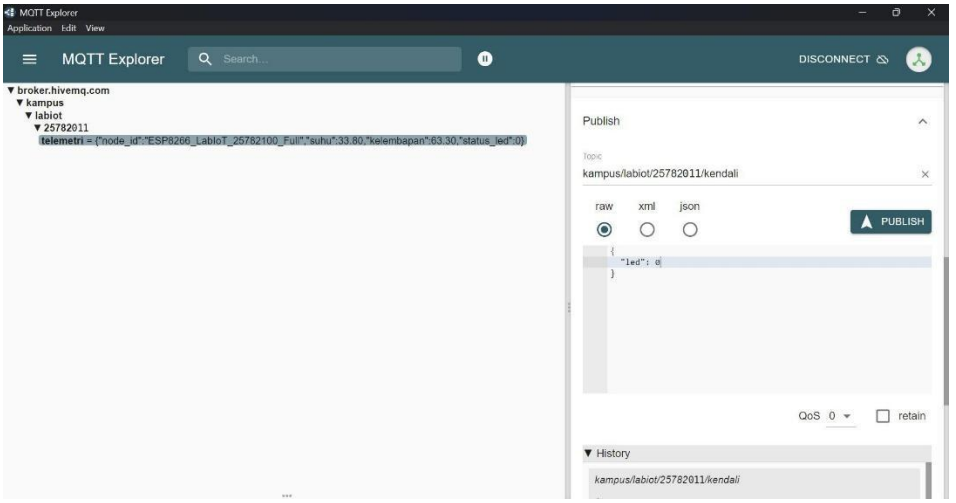
4. Menerbitkan Instruksi Kendali (Publish Command):
Arahkan ke panel Publish di sisi kanan bawah antarmuka.
Masukkan topik tujuan: kampus/labiot/nim_anda/kendali
Pilih format muatan: JSON.
Masukkan muatan payload berikut: **{ "led": 1 }**
Klik tombol Publish. Perhatikan bahwa LED indikator menyala seketika tanpa menghentikan siklus pengiriman data telemetri.



Hasil Output di Monitor :

```
13:24:32.717 -> [KONTROL DITERIMA] Topik: kampus/labiot/25782011/kendali | Payload: {
13:24:32.717 ->   "led": 1
13:24:32.717 -> }
13:24:32.717 -> --> [INDIKATOR] Status LED diubah: MENYALA
13:24:32.788 -> ----------------------------------------
13:24:33.745 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.50,"status_led":1}
13:24:37.737 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.40,"status_led":1}
13:24:41.702 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.50,"status_led":1}
13:24:45.702 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.70,"status_led":1}
13:24:49.746 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.80,"status_led":1}
13:24:53.729 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.50,"status_led":1}
13:24:57.724 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.70,"status_led":1}
13:25:01.702 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.60,"status_led":1}
13:25:05.719 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.60,"status_led":1}
13:25:09.743 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.60,"status_led":1}
13:25:13.705 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.50,"status_led":1}
13:25:17.701 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.70,"kelembapan":63.60,"status_led":1}
13:25:21.725 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.80,"kelembapan":63.80,"status_led":1}
```

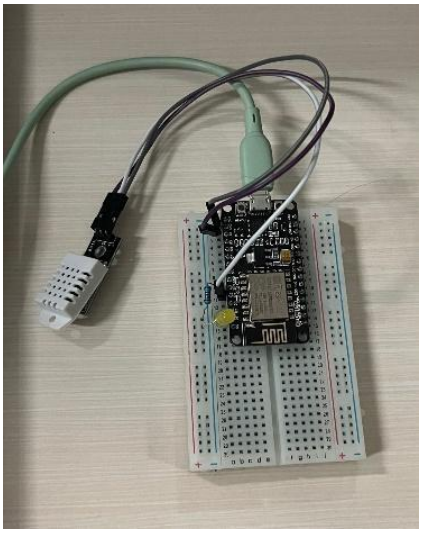
Kirimkan perintah pematian dengan payload {"led": 0} dan pastikan LED padam.



Hasil Output di serialmonitor:

```
13:27:57.775 -> ----------------------------------------
13:27:57.775 -> [KONTROL DITERIMA] Topik: kampus/labiot/25782011/kendali | Payload: {
13:27:57.775 ->   "led": 0
13:27:57.775 -> }
13:27:57.775 -> --> [INDIKATOR] Status LED diubah: PADAM
13:27:57.775 -> ----------------------------------------
13:28:01.701 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.80,"kelembapan":63.30,"status_led":0}
13:28:05.702 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.80,"kelembapan":63.50,"status_led":0}
13:28:09.701 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.80,"kelembapan":63.40,"status_led":0}
13:28:13.724 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782010_Full","suhu":33.80,"kelembapan":63.40,"status_led":0}
13:28:17.703 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.80,"kelembapan":63.30,"status_led":0}
13:28:21.702 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782011_Full","suhu":33.80,"kelembapan":63.40,"status_led":0}
```

Hasil perangkat atau rangkaian led Padam:



➢ Latihan

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):

Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

1. Efisiensi penggunaan kapasitas bandwidth pada paket data seluler.
2. Beban komputasi mikrokontroler (CPU overhead) saat menangani pertukaran data dua arah.
3. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (1-to N distribution)

Jawab:

1. Bandwidth. Pada MQTT, header tiap pesan kecil dan koneksi dibiarkan terbuka, sehingga biaya pembukaan sambungan tidak diulang. Pada HTTP biasa, tiap permintaan membawa header teks yang lebih besar, apalagi dengan token atau cookie. Efeknya terasa pada paket data seluler yang dikirim sering dan kecil. Perbandingan EMQX menyebut MQTT umumnya lebih hemat untuk pesan kecil dan sering.
2. Beban CPU mikrokontroler. Untuk dua arah, HTTP butuh polling: perangkat bertanya berulang untuk tahu ada perintah baru, tiap kali membuat permintaan dan memproses respons teks. Pada MQTT, perangkat cukup subscribe sekali, lalu broker mendorong pesan saat ada. Pustaka klien MQTT juga lebih kecil.
3. Distribusi 1-ke-N. Pada HTTP, server atau pengirim harus menghubungi tiap penerima satu per satu. Pada MQTT, publisher mengirim sekali ke broker, dan broker meneruskan ke semua subscriber yang cocok. Jumlah penerima bisa bertambah tanpa mengubah kode publisher.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):

Sebuah pabrik manufaktur memiliki 3 lini produksi (line1, line2, line3), di mana setiap lini memiliki 4 mesin (m1, m2, m3, m4), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (subscription topic) yang tepat menggunakan karakter wildcard (+ atau #) untuk:

1. Memantau parameter suhu dari seluruh mesin di semua lini produksi.
2. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.
3. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

Jawab:

1. Suhu semua mesin di semua lini:
   pabrik/+/+/suhu
2. Semua sensor mesin m1 di line2:
   pabrik/line2/m1/+
3. Semua lalu lintas pabrik:
   pabrik/#

Alasannya: + menggantikan tepat satu level, dan # harus di akhir serta mencakup semua level di bawahnya. Menurut dokumentasi Mosquitto, wildcard harus berdiri sendiri sebagai satu level penuh.

Latihan 3 (Implementasi Serialisasi Data Dinamis):

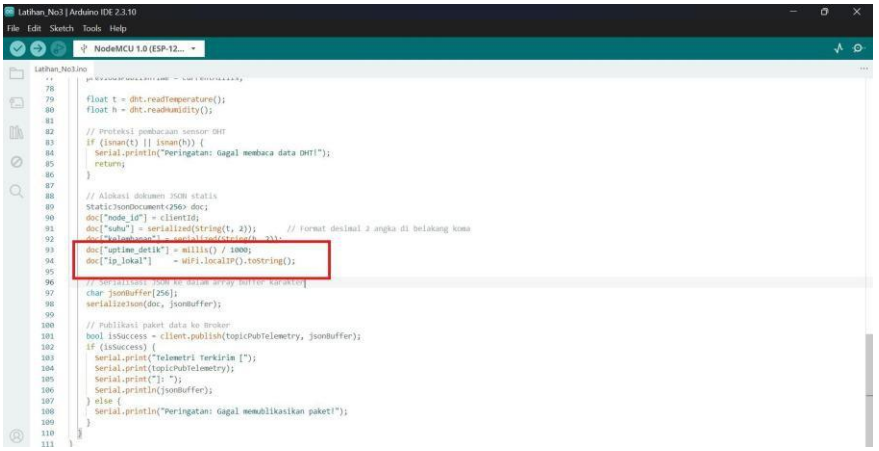
Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

1. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).
2. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).

Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

Jawab:

Bentuk Modifikasi:



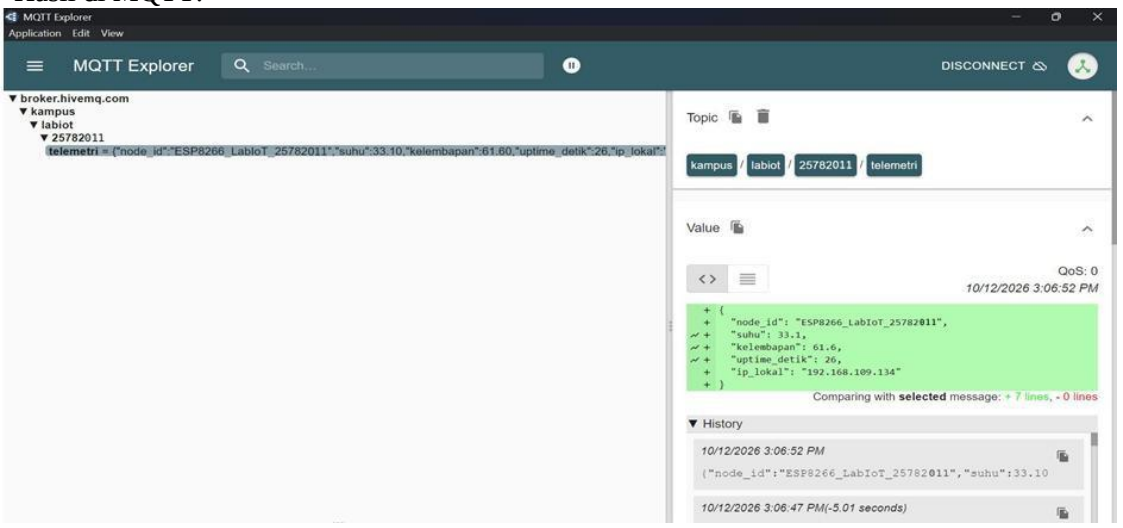
Penjelasan:

1. millis() menghitung waktu sejak NodeMCU mulai menjalankan program, dalam satuan milidetik.
2. millis() / 1000 mengubah waktu tersebut menjadi detik. Nilainya bertambah selama perangkat aktif dan kembali dihitung dari awal setelah perangkat di-reset.
3. WiFi.localIP() mengambil alamat IP lokal yang diberikan kepada NodeMCU oleh jaringan.
4. Wi-Fi.toString() mengubah alamat IP tersebut menjadi teks agar dapat dimasukkan ke JSON

Hasil Output:

```
15:06:29.715 -> ......
15:06:32.204 -> Wi-Fi Terkoneksi!
15:06:32.204 -> Alamat IP Lokal: 192.168.109.134
15:06:32.204 -> Menghubungkan ke MQTT Broker... Sukses Terhubung!
15:06:32.625 -> Telemetri Terkirim [kampus/labiot/25782011/telemetri]: {"node_id":"ESP8266_LabIoT_25782011","suhu":33.10,"kelembapan":61.60,"uptime_detik":6,"ip_lokal":"192.168.109.134"}
15:06:37.639 -> Telemetri Terkirim [kampus/labiot/25782011/telemetri]: {"node_id":"ESP8266_LabIoT_25782011","suhu":33.10,"kelembapan":61.60,"uptime_detik":11,"ip_lokal":"192.168.109.134"}
15:06:42.604 -> Telemetri Terkirim [kampus/labiot/25782011/telemetri]: {"node_id":"ESP8266_LabIoT_25782011","suhu":33.10,"kelembapan":61.60,"uptime_detik":16,"ip_lokal":"192.168.109.134"}
15:06:47.610 -> Telemetri Terkirim [kampus/labiot/25782011/telemetri]: {"node_id":"ESP8266_LabIoT_25782011","suhu":33.10,"kelembapan":61.60,"uptime_detik":21,"ip_lokal":"192.168.109.134"}
15:06:52.639 -> Telemetri Terkirim [kampus/labiot/25782011/telemetri]: {"node_id":"ESP8266_LabIoT_25782011","suhu":33.10,"kelembapan":61.60,"uptime_detik":26,"ip_lokal":"192.168.109.134"}
15:06:57.633 -> Telemetri Terkirim [kampus/labiot/25782011/telemetri]: {"node_id":"ESP8266_LabIoT_25782011","suhu":33.10,"kelembapan":61.50,"uptime_detik":31,"ip_lokal":"192.168.109.134"}
15:07:02.604 -> Telemetri Terkirim [kampus/labiot/25782011/telemetri]: {"node_id":"ESP8266_LabIoT_25782011","suhu":33.10,"kelembapan":61.50,"uptime_detik":36,"ip_lokal":"192.168.109.134"}
15:07:07.655 -> Telemetri Terkirim [kampus/labiot/25782011/telemetri]: {"node_id":"ESP8266_LabIoT_25782011","suhu":33.10,"kelembapan":61.60,"uptime_detik":41,"ip_lokal":"192.168.109.134"}
```

Hasil di MQTT:



Payload berhasil dibuat.

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):

Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah callback jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()? Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

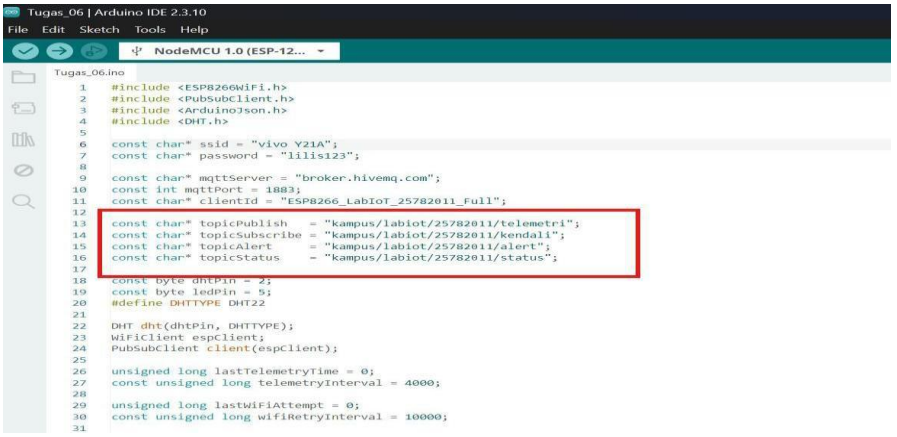
Jawab:

1. Dengan delay(6000) di loop(), client.loop() baru dipanggil sekali tiap 6 detik lebih. Pesan perintah yang tiba menunggu di buffer jaringan dan callback baru berjalan setelah delay selesai, jadi LED terlambat bereaksi sampai 6 detik. Telemetri dan pembacaan sensor juga ikut tertunda. Ada risiko lebih serius: broker memutus klien yang tidak mengirim paket dalam sekitar 1,5 kali interval keep-alive. Keep-alive bawaan PubSubClient adalah 15 detik. Dengan delay 6 detik biasanya masih aman, tetapi delay yang lebih panjang bisa membuat koneksi diputus. Karena itu kode memakai millis(), bukan delay().
2. reconnect() bekerja dengan perulangan while (!client.connected()). Ia mencoba client.connect(clientId). Kalau berhasil, ia langsung subscribe ulang ke topic kendali, penting karena sesi bersih (clean session) membuat langganan hilang setelah putus. Kalau gagal, ia mencetak kode client.state() lalu menunggu beberapa detik dan mencoba lagi. Untuk Wi-Fi yang putus, ESP8266 umumnya menyambung ulang sendiri, tetapi sesi MQTT-nya tetap harus dibuat ulang oleh fungsi ini.

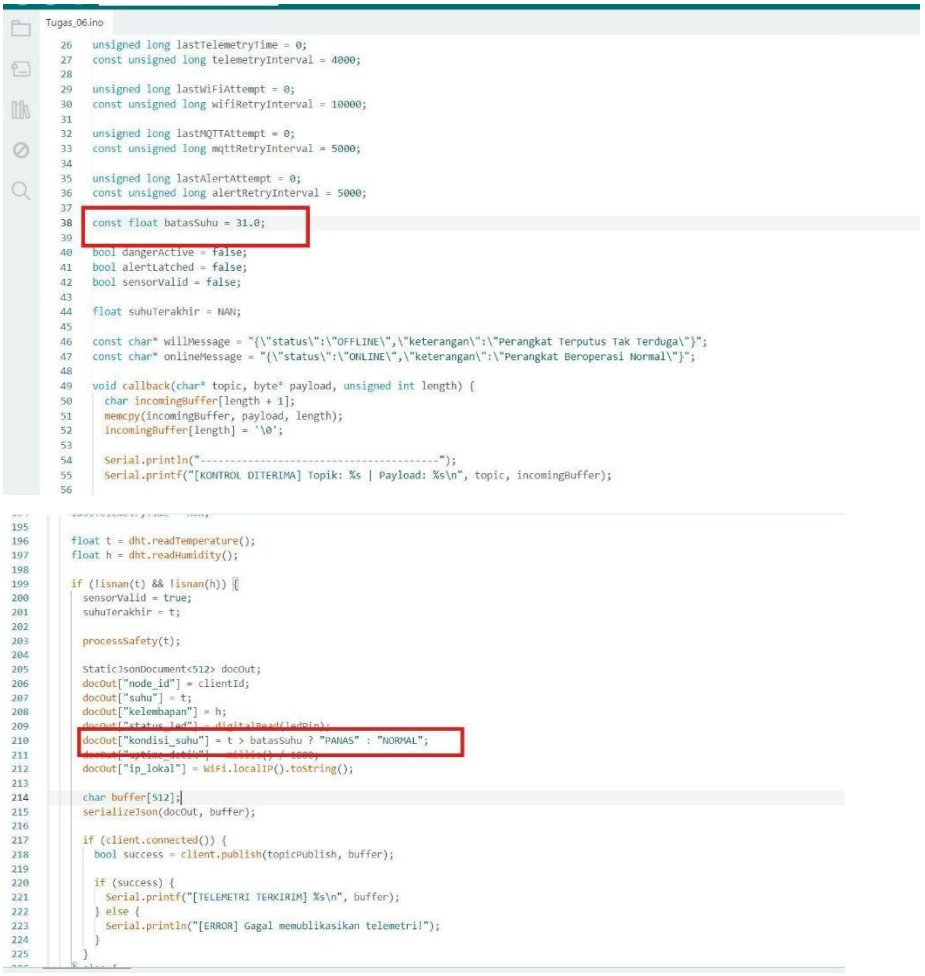
6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

Kode:

1. Konfigurasi topik dan pin — mengenalkan perangkat dan jalur komunikasinya.



2. Rule engine suhu — menjelaskan keputusan NORMAL atau PANAS.



3. Alert dan LED otomatis — menunjukkan respons saat kondisi bahaya



4. Uji pengaman kendali saat suhu bahaya



5. LWT — menjelaskan status ONLINE/OFFLINE.



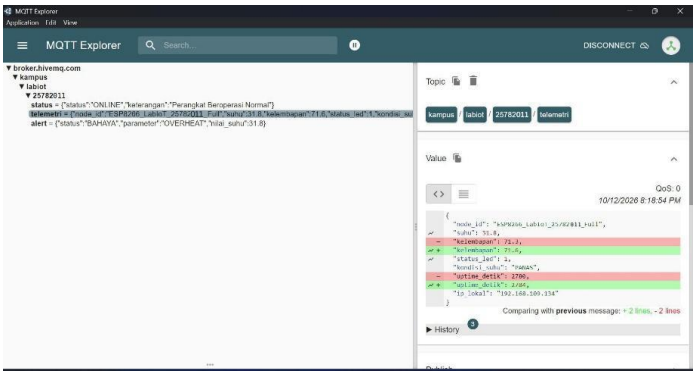
6. Uji perangkat tersambung kembali



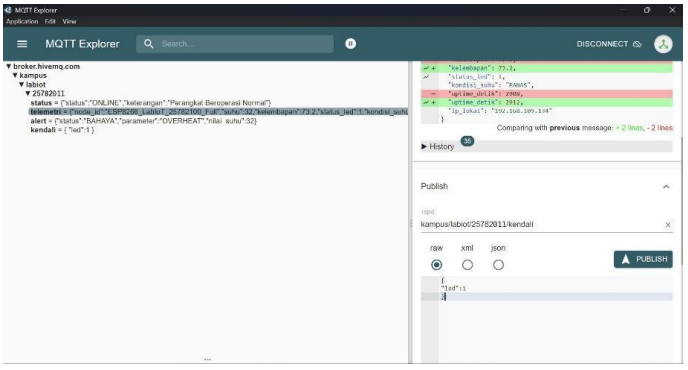
Hasil Output:



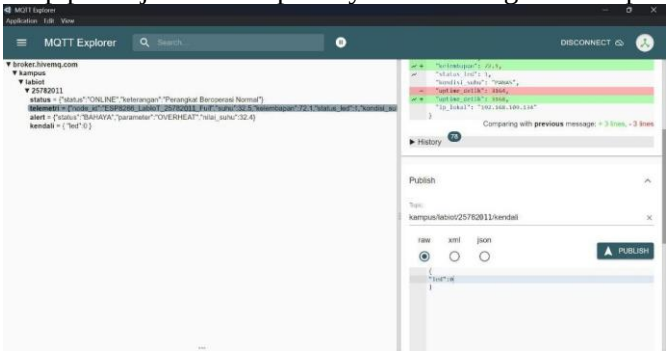
Hasil Output MQTT: Topic telemetri:



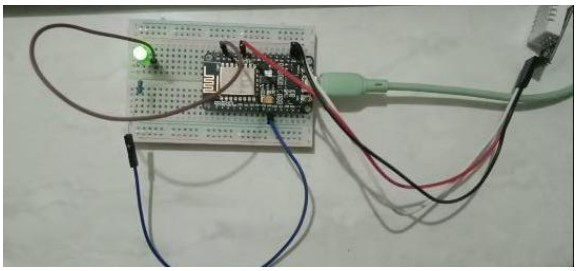
Topic Kendali mengubah menjadi led 1:



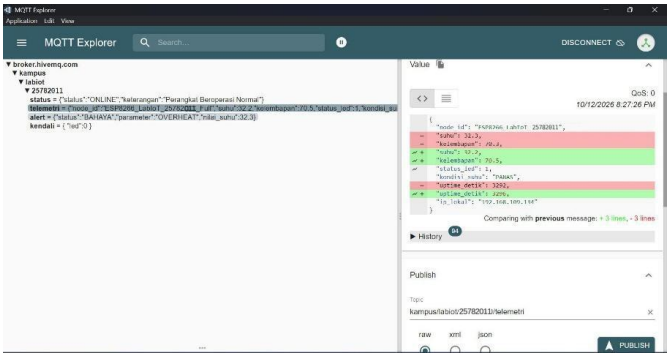
Saat topic kendali di ubah menjadi 0, led masih tetap menyala karena suhu di bandar lampung tetap panas jadi led tetap menyala dan mengeluarkan pesan alert bahaya:



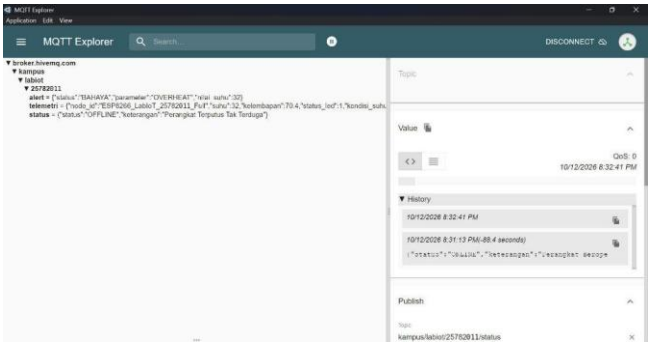
Bukti rnagkaian led tetap menyala:

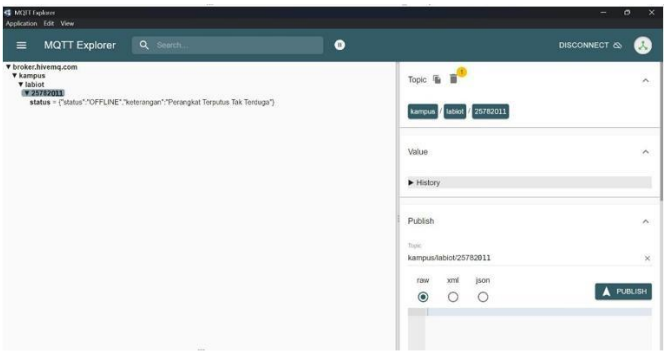


Hasil Topic Telemetri yang menampilkan pesan panas dan pesan alert bahaya:



Berikut hasil MQTT dari Ketika perangkat terputus mendadak dengan mencabut kabel daya:





➢ **Kesimpulan**

Kesimpulan dari praktikum ini adalah kita berhasil membuat sistem pintar untuk memantau suhu dan kelembapan ruangan secara otomatis menggunakan modul NodeMCU ESP8266, sensor DHT22, dan sistem pengiriman data MQTT.

Cara kerja dan poin penting dari sistem ini adalah:

• Pemantauan Otomatis: Sensor akan membaca suhu ruangan. Jika suhu masih di bawah 31℃, statusnya dianggap NORMAL. Namun, jika suhu sudah melebihi 31℃, sistem otomatis mengubah status menjadi PANAS.

• Sistem Keamanan Ketat: Begitu suhu terlalu panas (status OVERHEAT/BAHAYA), lampu LED peringatan akan langsung menyala otomatis. Selama ruangan masih panas, lampu ini tidak bisa dimatikan secara manual demi keselamatan.

• Laporan Cepat & Akurat: Data suhu dikirim rapi ke sistem pusat. Hebatnya, ada fitur Last Will yang bisa langsung memberi tahu kita apakah alat tersebut sedang aktif (ONLINE) atau tiba-tiba mati/rusak (OFFLINE).

• Program yang Lancar: Kode program dibuat cerdas tanpa perintah delay() (jeda waktu yang kaku). Hasilnya, alat bisa membaca suhu sambil mengirim data secara bersamaan tanpa ada drama nge-lag atau macet.